Port C and C++ courses to xeus-cpp for interactive learning

About Me

Name: Georgi Runtolev

GitHub: github.com/gRuntolev

Location: Plovdiv, Bulgaria

Email:

georgiruntolev_20v@schoolmath.eu

runtolev.georgi@gmail.com

Mentor: Vassil Vassilev

My name is Georgi Runtolev, and I am a student at High School of Mathematics “Akad. Kiril Popov” in Plovdiv. I have been passionate about programming for the past six years, with experience in C++ and C#. I am looking forward to further developing my skills and gaining practical experience through my internship at Compiler Research.

Project Overview

The xeus-cpp kernel provides a Jupyter interface for interactive C++ programming. Unlike traditional compilation workflows, xeus-cpp executes code cell by cell while

preserving the program state across executions. This allows students and developers to experiment, visualize, and debug C++ code incrementally, but introduces behaviors that are different from standard compilation, such as state persistence, redefinition conflicts, and execution order sensitivity.

The goal of this project is to port existing C and C++ course materials to xeus-cpp notebooks, making them interactive and suitable for hands-on learning. The project emphasizes understanding xeus-cpp's execution model, documenting differences from standard compilation, and producing teaching materials that help users explore C and C++ concepts in a notebook environment.

Project goals

Understand xeus-cpp's state management, including how variables, functions, and classes persist across cells.

Port C and C++ tutorial materials to xeus-cpp notebooks, covering topics such as:

- Fundamentals of C programming
- Modern C++ (C++11–C++20) features
- Memory management (pointers, arrays, dynamic allocation)
- Object-oriented programming and STL usage

Document the differences between notebook-based execution and traditional compilation workflows.

Create interactive notebooks that serve as teaching tools.

Test, debug, and provide guidelines for safe coding practices within xeus-cpp.

Produce a final report summarizing lessons learned, best practices, and recommended notebook structures for teaching.

Project timeline

Week	Activities	Deliverables	Milestone
1	Environment setup and basic verification.	Setup documentation.	Functional xeus-cpp.
2	Study state persistence across cells.	State behavior notebook.	Execution model understood.
3	Test redefinition and recompilation behavior.	Safe coding pattern guide.	Avoid kernel conflicts.
4	Port C fundamentals.	Interactive C basics notebook.	Functional C section.
5	Port C memory management and pointers.	Pointer lab notebook.	Low-level memory examples stable.
6	Port C++ OOP fundamentals.	OOP notebook.	Classes and objects working properly.

7	Port STL and modern C++ features.	STL lab notebook.	Modern C++ supported in xeus-cpp.
8	File I/O and exception handling.	I/O & error handling notebook.	Complete intermediate C++ coverage.
9	Advanced C++ (templates, inheritance).	Advanced notebook.	Stable complex C++ examples.
10	Multi-cell modular design testing.	Structured notebook template.	Verified scalable organization method.
11	Debugging experiments and stress testing.	Documentation of errors and edge cases.	Robust understanding of limitations.
12	Buffer week for fixing bugs, refactoring notebooks, and improving explanations.	Updated and cleaned notebooks.	Stable and polished content.
13	Create visual guides explaining state flow.	Diagrams and workflow explanations.	Teaching materials finalized.
14	Final testing and documentation refinement.	Final draft report.	Ready for submission.

15	Final report submission and repository organization.	Complete project package.	Project completed.